Ders 12 Genişletilmiş Notu: Genel Tekrar: Doğrulama ve Geçerleme, Test, Kalite Yönetimi, Konfigürasyon Yönetimi ve Süreç İyileştirme

Ders 12 Genişletilmiş Notu: Genel Tekrar: Doğrulama ve Geçerleme, Test, Kalite Yönetimi, Konfigürasyon Yönetimi ve Süreç İyileştirme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı: Farklı Bir Kaynaktan Genel Tekrar
- 2. Doğrulama ve Geçerleme
- 3. Yazılım İncelemesi (Inspection) ve Statik Analiz
- 4. Yazılım Testi
- 5. Test Case Tasarımı
- 6. Kalite Yönetimi
- 7. Yazılım Ölçümü ve Metrikler
- 8. Konfigürasyon Yönetimi
- 9. Süreç İyileştirme ve CMMI
- 10. Ders Sonrası Notlar
- 11. Örnek Sorular

1. Dersin Amacı: Farklı Bir Kaynaktan Genel Tekrar

Bu ders, dönemin proje sunumlarından önceki son dersidir. Hoca dönem boyunca işlenen konuları bu kez Sommerville'in *Software Engineering* kitabının özet slaytları (summary slides) üzerinden, "başka bir açıdan" tekrar etti. Tekrarın konuları şunlardır: doğrulama ve geçerleme (verification and validation), yazılım testi (software testing), yazılım kalitesi (software quality), konfigürasyon yönetimi (configuration management) ve sürüm yönetimi (release management). Hoca, kendi önceki slaytlarına göre bazı yeni eklemeler de yapılabileceğini söyledi. Anlatım sırasında eski derslerle bağlantılı ya da bağlantısız soruların yazılı veya sözlü sorulabileceğini belirtti. Dersi bir anlamda "farklı bir materyal üzerinden tekrar" olarak tanımladı.

2. Doğrulama ve Geçerleme

2.1 Konunun Çerçevesi

Hoca bu bölümde önce doğrulama ile geçerleme arasındaki farkı yeniden tartışacağını, sonra bunların süreç içindeki görevlerini, statik analizi, doğrulama ve geçerlemenin nasıl planlandığını, incelemelerin (inspection) ve otomatik statik analizin nasıl yapıldığını anlatacağını söyledi. Temiz oda (cleanroom) yazılım geliştirmeye ise ayrıntılı girmeyeceğini belirtti. Temiz oda dediğimizde, hata kritik yazılımların oldukça hassas bir ortamda geliştirildiği ve test edildiği, hatanın iyice azaltılması için kurulmuş bir yazılım geliştirme ortamından söz edilir.

2.2 Doğrulama ve Geçerlemenin Tanımı

Doğrulama (verification), "ürünü doğru yolla mı oluşturuyoruz?" sorusunun cevabıdır. Bunun için özellikle spesifikasyonları iyi takip etmek ve uygulamak gerekir. Geçerleme (validation) ise "doğru ürünü mü oluşturuyoruz?" sorusunun cevabıdır: ürün özellikleri doğru mu, istenen özellikler sağlanıyor mu?

Buradan şu sonuç çıkar: Doğrulama ve geçerleme, yazılım geliştirme yaşam döngüsünün her adımında mutlaka uygulanmalıdır. Hedef, sistemdeki hataları keşfetmek ve sistemin doğru yolla gerçekleştirildiğini görmektir. Hoca burada önemli bir ayrıma dikkat çekti: Amaç yazılımı tamamen hatalardan arındırmak değildir. Amaç, yazılımı *yeterince* hatalardan arındırıp arındırmadığımızı ve istenenlere uygun bir yazılım geliştirip geliştirmediğimizi görmektir. Yazılımın hedefleri sistem fonksiyonlarına, kullanıcı beklentilerine ve pazarlanacağı ortama uygun olmalıdır.

2.3 Statik ve Dinamik Doğrulama

- Yazılım incelemesi (software inspection) bir **statik doğrulamadır** (static verification). Program çalıştırılmadan, gerekirse araç desteğiyle yapılır.
- Test ise bir **dinamik doğrulamadır** (dynamic verification). Ürünün davranışı, özellikle çalıştırılarak gözlenir.

Hoca Sommerville'in kitabındaki şekil üzerinden, incelemenin hangi ürünlere uygulanabileceğini hatırlattı: gereksinim spesifikasyonu, yüksek seviye tasarım, formal spesifikasyon, ayrıntılı tasarım ve program. Bunların hepsi inceleme ile incelenebilir. Program testi ise yalnız program ve prototip üzerinde yapılır. Hoca program testinin yalnız geçerleme teknikleriyle değil, doğrulama amacıyla da kullanılabileceğini, kapsamın (coverage) yazılımdaki tüm hataların bulunmasını içine alabileceğini söyledi.

2.4 Kusur Testi ve Geçerleme Testi

Kusur testi (defect testing), sistemde hata olup olmadığını keşfetmeye yönelik testtir. Bu testte ne kadar çok hata bulunursa test o kadar başarılıdır. İlk aşamalardaki testlerde, yani daha çok doğrulamaya hizmet eden birim testi, bütünlük testi ve bileşen (component) testinde kusur testi yapılır.

Geçerleme testinde (validation testing) ise gereksinimlerin karşılanıp karşılanmadığına bakılır. Fonksiyonel olmayan gereksinimlere daha çok sistem testinde bakılır. Kullanım (kullanıcı) testinde de geçerleme testi yapılır.

2.5 Test ve Debug

Hoca kusur testi ile debug'ın ayrı süreçler olduğunu vurguladı. Test hatanın varlığını ortaya koyar. Debug ise hatanın yerini bulma ve bu hatayı düzeltme işlemidir. Hoca debug kavramından daha önce bahsettiğini, ama debug sürecinin adımlarına bakmadıklarını söyledi. Süreç şöyledir:

```
Test sonuçları (program test case'lerle test edilmiş)
  -> Hatanın yerini bul
  -> Hata onarımını tasarla (spesifikasyonlar üzerinden giderek)
  -> Hatayı onar
  -> Programı yeniden test et (yine test case'ler vererek)
```

2.6 Doğrulama ve Geçerlemenin Planlanması

Nasıl yazılım geliştirme için plan yapılıyorsa, doğrulama ve geçerleme için de plan yapılır. Bu plan geliştirme sürecinde hazırlanır. Planda statik doğrulama (inceleme) ile dinamik doğrulama (test) arasında bir denge kurulmaya çalışılır. Test planı ayrıca test sürecinin standartlarını belirler.

Hoca daha önce dikey olarak gördükleri V modelinin burada yatay olarak çizildiğini gösterdi. Geliştirme tarafında sırasıyla gereksinim spesifikasyonu, sistem spesifikasyonu, sistem tasa-

rımı ve ayrıntılı tasarım yapılır, ardından modülün kodu oluşturulur. Şeklin ortasında, hangi geliştirme aşamasında hangi test planının hazırlandığı görülür:

Geliştirme aşaması	Bu aşamada hazırlanan plan	İlgili test
Gereksinim spesifikasyonu	Kabul testi planı	Kabul testi
Sistem spesifikasyonu	Sistem bütünlük testi planı	Sistem bütünlük testi
Sistem tasarımı	Alt sistem bütünlük testi planı	Alt sistem bütünlük testi
Ayrıntılı tasarım, modül kodu	-	Modül ve birim testi

Geliştirme süreci modül kodlanarak tamamlanır. Ardından test sırası şöyledir: modül/birim testi (unit test), alt sistem bütünlük testi, sistem bütünlük testi ve kabul testi. Kısaca birim testi, bütünlük testi, sistem testi ve kabul testi.

2.7 Test Planının Yapısı

Hoca, öğrencilerin projeleri için hazırladığı test planlarını hatırlatarak Sommerville'deki test planı yapısını anlattı. Daha önce başka bir test planı tanımı verdiğini, ikisinin birbirine benzediğini ve hangisinin takip edileceğinin serbest olduğunu söyledi. Test planında şunlar bulunur:

- Test süreci: test nasıl yürüyecek?
- Gereksinim izlenebilirliği (requirements traceability): gereksinimler nasıl izlenecek?
- Test edilecek parçalar.
- Test takvimi (schedule), örneğin bir Gantt diyagramı ile.
- Test kayıt süreçleri.
- Donanım ve yazılım gereksinimleri.
- Test sürecindeki kısıtlar.

Kayıt konusunda hoca şunu vurguladı: Testleri yalnız çalıştırıp sonuçlarını göstermek yeterli değildir. Sonuçların raporlanması ve sistematik olarak kaydedilmesi gerekir. Test sürecinin doğru yürütüldüğünün kayıt altına alınması ve denetlenebilmesi gerekir. Donanım ve yazılım gereksinimleri kısmında test için bir yazılım aracı (tool) ya da donanım kullanılıp kullanılmadığı, kısıtlar kısmında da test sürecinde bir kısıt olup olmadığı belirtilir.

3. Yazılım İncelemesi (Inspection) ve Statik Analiz

3.1 İncelemenin Başarısı ve Testle İlişkisi

İnceleme, yazılımın sistemimize uygun olup olmadığının incelenmesi ve hatalarının ortaya konmasıdır. Hoca incelemenin neden başarılı olduğunu anlatırken tek bir incelemede birçok hatanın birlikte fark edilebileceğini söyledi. Testte ise bir modül ya da birim test edilirken, onunla ilişkili başka birimler ve bunlarla ilişkisi de çalışma sırasında test edilmiş olur.

Hoca buradan önemli bir sonuç çıkardı:

> **Hocanın vurgusu:** "Ya inspection yapacağız ya test yapacağız" diyemeyiz. İnceleme ve test birbirinin alternatifi değildir. Doğrulama ve geçerleme sırasında ikisi birbiriyle bütünleşik olarak, birlikte gerçekleştirilmelidir.

İkisinin farkı şöyle özetlenir: Ürünün spesifikasyonla örtüşüp örtüşmediğine incelemede bakılabilir. Ama müşterinin gerçek gereksinimleriyle örtüşüp örtüşmediğine testle bakılır. Örneğin performans ve kullanılabilirlik gibi işlevsel olmayan özelliklere incelemeyle bakılamaz. Bunlar ancak sistem çalıştırılıp kullanıcıya gösterildiğinde, yani testle gösterilebilir.

İncelemede kod hataları ve tasarım hataları takip edilerek, ürün standartlarla eşleştirilerek hatalar, anomaliler, mantık hataları ve standartlarla örtüşmeyen yerler bulunabilir.

3.2 İncelemenin Ön Koşulları

- Ayrıntılı bir spesifikasyon olmalıdır.
- İncelemeyi yapacak ekip üyeleri kurum standartlarına aşina olmalıdır.

- Hataların tanımlanması için önceden kontrol listeleri (checklist) hazırlanmalıdır.

Hoca kontrol listesini kendi üzerinden örnekledi: Öğrencilerin hazırladığı bir gereksinim analizi belgesini, spesifikasyonunu inceleyecek olsa, “nerelerde hata olabilir, şurası doğru mu, şu standarda uygun mu” maddelerinden oluşan bir kontrol listesi hazırlayıp belgeyi ona göre değerlendirebilir.

3.3 İnceleme Süreci ve Roller

İnceleme süreci de planlanır. Adımları planlama, hızlı gözden geçirme (overview), bireysel hazırlık, inceleme toplantısı ve yeniden çalışmadır (rework). İnceleme bir topluluk içinde yapılır: Önce herkes kendisine görev olarak verilen modülleri ayrı ayrı inceler. Sonra herkesin üstlendiği modüllerin birlikte gözden geçirildiği bir inceleme toplantısı yapılır. Hoca daha önce anlatılan Fagan incelemesini hatırlattı: Fagan incelemesinde ekiplerin kuruluşu ve süreçteki adımlar daha ayrıntılıdır.

Rol	Görev
Yazar	Programı ya da tasarımı yazan programcı veya tasarımcı
İnceleyici	Hataları bulmakla görevli kişi
Okuyucu	İnceleme toplantısında incelenen kodların hepsini toplu gözden geçirmek için okuyan kişi
Moderatör	Süreci yöneten kişi

Bu görevler uygulama ortamının ve grubun yapısına göre dağıtılır. Her role ayrı bir kişi atanabileceği gibi, bir kişi birden fazla rolü de üstlenebilir.

3.4 Kontrol Listeleri

Hoca, bir bölümün incelemesinde hangi hatalar daha sık çıkıyorsa bunların maddelenmesinin, sonradan bunlar üzerinde istatistik yapmayı da kolaylaştıracağını söyledi. Kontrol listeleri genel başlıklarla ve alt başlıklarla kurulabilir:

- Veri hataları
  - İsimlendirme hatası var mı?
  - Karakter dizisi (string) kullanımında hata var mı?
  - Taşma (overflow) var mı?
- Kontrol hataları
  - Koşul ifadesi doğru mu?
  - Sonsuz döngü var mı?
- Girdi-çıktı hataları
- Ara birim hataları
- Bellek yönetimi hataları

3.5 İnceleme Hızları ve Maliyet

Hoca bir kişinin incelemesinde saatte yaklaşık 125 kod ifadesinin incelendiğini söyledi. Sürecin aşamalarına göre hızlar şöyledir: Önce planlama yapılır. Sonra çok hızlı bir gözden geçirme yapılır, bunun ardından her inceleyiciye ayrıntılı inceleyeceği modüller atanır. Herkes kendi kısmını inceledikten sonra toplantıda birlikte üstünden geçilir.

Aşama	Hız (saatte ifade)
Hızlı gözden geçirme	Yaklaşık 500
Bireysel hazırlık (görev verilen bölümün ayrıntılı incelenmesi)	Yaklaşık 125
İnceleme toplantısı (birlikte tek tek geçilir)	Yaklaşık 125

Her şeyin çalıştırılmadan, standartlar ve mantık açısından tek tek gözden geçirilmesi gerektiği için inceleme pahalı, maliyetli bir süreçtir. Hoca bunun yaklaşık 40 adam-saatlik bir miktara denk geldiğini söyledi.

> **Ders sonrası not:** Altyazıda 40 adam-saatin hangi kod miktarı için olduğu söylenmiyor. Sommerville'de bu değer büyük olasılıkla yaklaşık 500 satırlık bir programın incelenmesi için verilir. Hocanın verdiği hızlarla bir kişi 500 ifade için $500/500 + 500/125 + 500/125 = 1 + 4 + 4 = 9$ saat harcar (ara adım eklendi). Dört kişilik bir ekipte bu $4 \times 9 = 36$ adam-saat eder, yani yaklaşık 40 adam-saat.

3.6 Otomatik Statik Analiz

Bu maliyetli analizi daha az maliyetli hale getirmek için yazılım araçları vardır. Statik analiz araçları programı inceler ve olası hata koşullarına doğrulama ve geçerleme ekibinin dikkatini çeker. Hoca bu yola gidilebileceğini, genellikle de gidildiğini söyledi. Statik analizin aşamaları şunlardır:

- Kontrol akışı analizi.
- Veri kullanım analizi.
- Ara birim analizi.
- Yol analizi: bütün program yollarının ve kontrol deyimlerinin araştırılması.

Slaytta bir analiz aracı örnek verilmişti, ama hoca kaynağın eski olduğunu, günümüzde çok sayıda farklı araç bulunduğunu söyledi. Formal metotların da kullanılabileceğini belirtti, fakat bu kadar ayrıntıya girmek istemediğini ve formal metotlara karşı başka seçeneklerin de kullanılabileceğini ekledi.

3.7 Temiz Oda (Cleanroom) Yazılım Geliştirme

Formal spesifikasyon yapılarak statik doğrulamanın daha ağırlıklı kullanıldığı ve program güvenilirliğini artıran bir süreç izleniyorsa, temiz oda yazılım geliştirmeden söz edilir. Bu süreçte hata bulma ve formal spesifikasyon üzerinden hata bulma ağırlıklıdır. Hedef güvenilirliğin (reliability) yüksek olmasıdır. Statik doğrulama ve incelemeler temiz oda süreçlerinde çok ağırlıklı kullanılır. Hoca bu konuya daha fazla girmedi.

> **Ek bilgi (derste anlatılmadı):** Temiz oda adı, büyük olasılıkla yarı iletken üretimindeki toz ve kirden arındırılmış "temiz oda" fikrinden gelir: Amaç hatayı sonradan ayıklamak değil, hatanın hiç girmesini önlemektir. Bu yaklaşımda artımlı geliştirme ve istatistiksel kullanım testinin de yer aldığı bilinir.

4. Yazılım Testi

4.1 Bölümün Kapsamı

Hoca bu bölümde geçerleme testi ile kusur testi arasındaki farkları, sistem ve bileşen testini, test case'lerin hangi stratejiyle üretildiğini ve test otomasyonu için kullanılan araçları ele alacağını söyledi.

4.2 Bileşen Testi ve Sistem Testi

Bileşen testi (component testing), aslında birim testi ve onun bir kademe üstündeki bileşenlerin, yani tek bir program parçasının test edilmesidir. Bu testten genellikle geliştirici (developer) sorumludur ve test geliştiricinin deneyimine dayanır.

Sistem testi ise birden fazla bileşenin veya alt sistemin bütünleşmesinden oluşan bir gruba uygulanır. Bu testten bağımsız bir test ekibinin sorumlu olması daha uygundur. Sistem testinde sistem spesifikasyonları test edilir. Bileşen testinden çıkan birden fazla birim önce bütünlük testiyle birleştirilir, ardından sistem testi uygulanır. Şekilde de görüldüğü gibi yazılım geliştirici bileşen testinden ve birim testinden sorumludur.

4.3 Kusur Testi ve Geçerleme Testi

	Geçerleme testi	Kusur testi
Amaç	Geliştiriciye ve sistem müşterisine gereksinimlerin karşılandığını göstermek	Programdaki, spesifikasyona uymayan hatalı işlemleri keşfetmek
Başarılı test	Gereksinimlerin geçerliğinin sağlandığını gösterir	Doğru olmayan, hatalı yerleri keşfeder; ne kadar çok hata bulunursa o kadar başarılıdır

4.4 Test Süreci

Test süreci şu adımlarla ilerler:

```
Test case tasarımı -> Test case'ler -> Test verisinin hazırlanması
  -> Programın test verisiyle çalıştırılması -> Test sonuçları
  -> Sonuçların beklenen sonuçlarla karşılaştırılması -> Test sonuç raporu
```

Sistem testinde uyulacak politikalar, yani testlerin nasıl seçileceği şöyledir:

- Menülerden ulaşılabilen tüm fonksiyonlar denenir.
- Birlikte çalışan fonksiyon kombinasyonları denenir.
- Kullanıcı bir girdi verdiğinde, bu girdiye göre elde edilen sonucun doğru olup olmadığı test edilir.

4.5 Bütünlük Testi

Sistem testinden önce bütünlük testi (integration testing) yapılır. Alt sistemler denendikten sonra bütünlük testinde test ekibi bütün kaynak koda erişebilir ve sistem bir bütün olarak test edilir.

Bütünlük testi yukarıdan aşağıya (top-down) ya da aşağıdan yukarıya (bottom-up) yapılabilir. Her iki durumda da sistem artımlı (incremental) olarak birleştirilir. Artımlı bütünlük testinde test sırası T1, T2, T3 şeklindedir: Her yeni birim eklendiğinde testler birleştirilerek yeniden çalıştırılır. Altta daha çok ya da daha az birim olabilir, yani bütünleştirme farklı test dizileri üzerinden yapılabilir.

Yukarıdan aşağıya bütünlük testi, sistem mimarisindeki hataları keşfetmek için daha iyidir. Aşağıdan yukarıya yaklaşımda ise önceden test edilmiş modüller birleştirilerek adım adım yukarı çıkılır.

4.6 Sürüm Testi (Release Testing)

Sürüm testi, müşterilere hizmete sunulacak sürümün yeniden bütün olarak test edilmesidir. Dağıtımdan önce, yeni bir sürüme geçileceği zaman her şeyin yolunda olduğunu ve kullanıcının istediğinin gerçekleştirildiğini gösterir. Hoca sürüm yönetiminde bunun planlanması ve iyi yönetilmesi gerektiğini ekledi. Sürüm testi genellikle kara kutu (black box) testi, yani fonksiyonel testtir ve müşteri spesifikasyonlarına dayanır.

4.7 Kara Kutu Testi

Kara kutu testinde test edilecek sistemin iç yapısı hakkında bilgimiz olmayabilir. Ama sistemin ne yapması gerektiğini biliriz. Girdileri verip uygun çıktıları alıp alamadığımıza bakarız. Test sırasında bazı kurallara uyulmalıdır:

- Hata mesajlarına bakılmalıdır (hata mesajlarını tetikleyecek girdiler denenmelidir).
- Aynı girdilerle birkaç kez çalıştırılmalıdır.
- Geçersiz veri girilip çıktıları kontrol edilmelidir.

4.8 Senaryo ve Use Case Tabanlı Test

Slayttaki örnek bir kütüphane sistemidir. Önce bir test senaryosu oluşturulur: Sistemin nasıl işleyeceği kullanıcının ağzından anlatılır. Bu senaryodan, kullanıcı testinde hangi işlemlerin bulunması gerektiği çıkarılır. Kütüphane örneğinde test edilmesi gerekenler şunlardır:

- Doğru ve hatalı kullanıcı bilgilerini ayırt eden bir giriş (login) mekanizması.
- Sistemin belgeler hakkında doğru bilgiyi çıkarıp çıkarmadığı.
- Bilginin indirilebilmesi (download) için gerekli izin mekanizmasının olup olmadığı.

Hoca, use case'lere bakıp bir use case'in nasıl test edileceğinin çıkarılabilmesi gerektiğini söyledi. Örneğin ardışıl diyagrama (sequence diagram) bakarak bir use case'in tanımında hangi testlerin yapılacağı bulunabilmelidir.

4.9 Performans Testi ve Stres Testi

Performans testi, sürümden önce yapılması gereken sistem testlerinden biridir. Bu testte cevap süresi ve ayrıca güvenilirlik test edilir. Stres testinde (dayanıklılık testi) ise sistemin taşıyabileceği en yüksek yük bulunur ve bu yükte hata olup olmadığına bakılır. Dağıtık sistemlerde ağın ve kullanıcı sayısının dayanıklılığı test edilir.

Soru: Derste sohbet kısmından bir web uygulamasında performans testinin nasıl yapılabileceği soruldu. **Cevap:** Hoca, web uygulaması için performans testinin anlatıldığı şekilde yapılabileceğini ve sonuçlarının rapora aktarılabileceğini söyledi.

4.10 Bileşen Testi

Hoca bileşen testini birim testin bir kademe üstü olarak tanımladı ve bileşen testi ile birim testinin aynı şeye getirilebileceğini söyledi. Bu süreç bir kusur testidir. Her bileşende tek tek bütün işlemler, fonksiyonlar, sınıf tanımları, özellikler, metotlar ve kullanıcı ara birimleri işlevsel mi ve doğru çalışıyor mu test edilir. Nesnelerin ve sınıfların doğru oluşturulduğu da aynı zamanda test edilmelidir. Slaytta bir hava durumu (weather station) uygulamasının test işlemi örnek olarak verilmişti.

4.11 Ara Birim Testi

Ara birim testi (interface testing), girdilerin doğru aktarıldığını ve kullanıcılarla etkileşimin doğru olduğunu gösteren testtir. Hem modüller arasındaki ilişkiler hem de kullanıcı ile sistem arasındaki ara birimler test edilir.

Ara birim türleri:

- Parametre geçişi.
- Paylaşımlı bellek ara birimleri.
- İşlemsel (prosedürel) ara birimler, alt sistemler arası geçişler.
- Mesaj geçişi.

Ara birim hataları:

- Ara birimin eksik olması.
- Parametrelerin yanlış sırada verilmesi.
- Ara birimin mantıksal olarak yanlış hazırlanması.
- Zamanlama hataları.

Ara birimlerin nasıl test edileceği:

- Parametreler uç (ekstrem) değerlerde ve aralık sınırlarında seçilerek testler tasarlanabilir.
- Bileşenin hatalı çalışmasına neden olacak testler tasarlanabilir.
- Mesaj geçişli sistemlerde stres (dayanıklılık) testleri kullanılabilir.

5. Test Case Tasarımı

Test case tasarımının amacı, etkin bir geçerleme ve kusur testi yapacak test case'ler üretmektir. Test case'ler üç yaklaşımla tasarlanabilir: gereksinim tabanlı test, bölümleme testi (partition testing) ve yapısal test (structural testing).

5.1 Gereksinim Tabanlı Test

Gereksinim tabanlı testte, gereksinim spesifikasyonundaki gereksinimlerin sağlanıp sağlanmadığı test edilir. Hoca öğrencilerin projelerinde bu testleri oluşturacağını hatırlattı. Örnek: Bir veri tabanında kullanıcı arama işlemi. Mevcut bir kullanıcı ve mevcut olmayan bir kullanıcı arandığında sistemin hangi yanıtları verdiğine bakılabilir. Gereksinimlerde bulunan işlemlerin nasıl uygulandığı bu şekilde test edilir.

Hoca bu noktada 15 dakikalık ara verdi ve aradan sonra bölümleme testinden devam etti.

5.2 Bölümleme Testi (Equivalence Partitioning)

Bölümleme testi kara kutu test tiplerinden biridir. Girdi verisi ve çıktı sonuçları farklı sınıflara bölünerek test edilir. Denklik bölümlemesinde (equivalence partitioning) geçerli girdiler de geçersiz girdiler de sınıflara bölünür. Böylece hangi aralıktaki girdilere hangi çıktıların alındığı, geçersiz girdilere nasıl bir sonuç alındığı görülür.

Bu yöntemin kara kutu testinde kullanılmasının nedeni şudur: İç yapı bilinmediği için test verisi iç yapıya göre belirlenemez. Veri dış ortamdan girilirken sınıflar üzerinden seçilir.

Hoca bir örnek üzerinden girdi değerlerinin belli aralıklara bölündüğünü anlattı. Az test yapma imkânı varsa girdiler üç gruba bölünür. Daha çok test yapılabiliyorsa daha küçük aralıklara bölünüp test girdileri bunların içinden seçilir. Aralıklar oluşturulurken yapılabilecek test sayısı göz önüne alınır ve test sayısına göre her aralıkta bir değer olacak şekilde test verisi oluşturulur.

Büyük olasılıkla slayttaki örnek şöyledir:

Girdi	Sınıflar	Seçilen test değerleri
Girdi değeri sayısı	4'ten az, 4-10 arası, 10'dan çok	3, 4, 7, 10, 11
Girdi değerleri	10000'den küçük, 10000-99999 arası, 99999'dan büyük	9999, 10000, 50000, 99999, 100000

Aynı yöntem bir arama rutininin (ikili arama, binary search) test verisini oluşturmak için de kullanılır.

> **Ek bilgi (derste anlatılmadı):** Denklik sınıflarının değerleri, sistemin bir sınıftaki bütün değerlere aynı biçimde davranması beklenecek şekilde seçilir. Bu yüzden her sınıftan bir temsilci yeterli kabul edilir. Hatalar sınıf sınırlarında sık görüldüğü için sınır değer analizi (boundary value analysis) ile birlikte kullanılır. Tablodaki 3, 4, 10, 11 ve 9999, 10000, 99999, 100000 değerleri bu sınırlardan seçilmiştir.

5.3 Yapısal Test

Yapısal test (structural testing), saydam kutu testi ve beyaz kutu testi (white box testing) aynı anlamda kullanılır. Test case'ler program yapısına göre seçilir ve her ek test case programın yeni bir dalını takip edecek şekilde seçilir. Amaç tüm program ifadelerini, yani tüm temel yolları test etmektir. Yapısal testte test verisi birim kodu üzerine gönderilir ve test çıktıları elde edilir.

5.4 Yol Testi: İkili Arama Örneği

İkili arama (binary search) için yol testinde önce algoritmanın akış grafı (flow graph) çizilir. Akış grafında kaç tane kapalı bölge varsa, buna bir eklenerek temel (bağımsız) yol sayısı bulunur. Bu sayı iç yapıdan ortaya çıkar. Hoca şekil üzerinde "bir kapalı alan, iki kapalı alan, üç kapalı alan, bir de dördüncü olarak dış alan" diyerek dört temel yol olduğunu gösterdi.

```
1 -> 2 -> 3 -> 4 -> 5
5: bottom > top          -> 14 (çıkış)
5: while bottom <= top   -> 6 -> 7
7: elemArray[mid] = key  -> 8 -> 9 -> 10 -> 14
7: elemArray[mid] != key -> 11
11: elemArray[mid] > key -> 12 -> 5
11: elemArray[mid] < key -> 13 -> 5
```

Bağımsız yollar:

- 1, 2, 3, 4, 5, 14: döngüye hiç girilmez.
- 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 14: anahtar bulunur.
- 1, 2, 3, 4, 5, 6, 7, 11, 12, 5, ...: aramaya bir yarıda devam edilir.
- 1, 2, 3, 4, 5, 6, 7, 11, 13, 5, ...: aramaya öbür yarıda devam edilir.

Test case'ler bu yollardan türetilir. Her yol en az bir kez çalıştırılmalıdır. Yolların gerçekten çalışıp çalışmadığını görmek için bir dinamik program analizcisi kullanılabilir.

> **Ders sonrası not:** Hocanın "kapalı bölge sayısı + 1" kuralı, çevrimsel karmaşıklık (cyclomatic complexity) formülü $V(G) = E - N + 2$ ile aynı sonucu verir. Yukarıdaki grafta $E = 16$ kenar ve $N = 14$ düğüm vardır: $V(G) = 16 - 14 + 2 = 4$. Bu, $3 + 1 = 4$ temel yol sayısıyla örtüşür.

5.5 Test Otomasyonu

Artık birçok büyük yazılımda, test verisini hazırlama ve çalıştırma sürecini kısaltmak, hızlandırmak ve maliyeti azaltmak için test otomasyonu kullanılır. Örnek olarak Java için JUnit verildi. Hoca öğrencilerin çoğunun projeleri için test araçlarını zaten araştırmış olduğunu düşündüğünü söyledi. Test araçları test betikleri (script) oluşturmakta, kaynak kod üzerinde dinamik analiz yapmakta ve test raporu üretmekte kullanılır.

6. Kalite Yönetimi

6.1 Kalite ve Kalite Yönetimi

Hoca bu bölümde kalite yönetimini, ardından yazılım süreç iyileştirmeyi ve konfigürasyon yönetimini ele alacağını söyledi. Kalite yönetimi bir kurumda bir kalite yönetim ekibiyle sağlanır. Daha önce kaliteyi sağlamak için hangi aktivitelerin geçerli olduğu ve kalite özelliklerinin neler olduğu üzerinde durulmuştu. Bu derste konuya daha geniş bir açıdan bakıldı: kalite yönetimi aktiviteleri, kalite standartları, kaliteyi kontrol altına almak için yazılım metrikleri ve yazılım ölçümü.

Bir yazılım ürününün kalite özellikleri daha önce McCall modeline göre ayrıntılı incelenmişti. ISO 9126'da da kalite özellikleri tanımlıdır. Kurumda kalite standartlarını takip edecek bir ekip olmalıdır. Ayrıca bir kalite kültürü bulunmalıdır: Ekipte kim, kime karşı, hangi kaliteden sorumludur?

Kalite, yazılımın kullanımı sırasında müşterinin ve geliştiricinin bakış açısıyla sağlaması gereken özellikler ve bunların sağlanma derecesidir. Tanımlanan spesifikasyonlara uygunluk ve bu uygunluğun derecesi de kalite yönetimi içinde tanımlanmalıdır: Gereksinim spesifikasyonunda belirtilenler ne derece karşılanıyor?

Kalite yönetimi büyük ve karmaşık sistemler için daha fazla önem taşır. Bu sistemlerde süreç boyunca takip edilecek ve bütün geliştirme ekibine uygulatılacak bir kalite belgelemesi vardır. Daha küçük sistemlerde kalite yönetimi daha az belgeyle ve kalite kültürünü sağlayacak daha az kişiyle yürütülür.

6.2 Kalite Yönetimi Aktiviteleri

Aktivite	Açıklama
Kalite güvencesi (quality assurance)	Kalite için hangi standartların ve süreçlerin kurulacağının belirlenmesi
Kalite planlama	Belirlenen standardı uygulamak için her projede ayrı bir kalite planı yapılması
Kalite kontrolü (quality control)	Yazılım geliştirme ekibinin uygulaması gereken standartlara uyulduğunun kontrolü

Kalite yönetim ekibi, yazılım geliştirme süreçleri içinde bu süreçleri takip etmekle görevlidir.

6.3 Süreç Kalitesi ve Ürün Kalitesi

Süreç kalitesi, yazılım geliştirme ekibinin işleri kurallara uygun yapıp yapmadığının ve hazırladığı belgelerin standartlara uygun olup olmadığının takip edilmesidir. Kısaca: Yazılım geliştirme süreci kurallara uygun gidiyor mu? Ürün kalitesi ise ortaya çıkan yazılım ürününün istenen gereksinimleri ve kalite özelliklerini sağlayıp sağlamadığının kontrol edilmesidir.

Süreç tabanlı kaliteden söz edildiğinde, süreç ile ürün arasındaki ilişkiye bakılır. Şelale (waterfall) modeline göre gereksinim analizi, tasarım, gerçekleştirme ve test aşamalarının her birinde yazılım mühendisliği kurallarına uygun hareket ediliyor mu? Süreç tabanlı kalitenin akışı şöyledir:

```
Süreci tanımla -> Ürünü geliştir -> Ürün kalitesini değerlendir
  kalite tamam    -> süreç uygun yürütülmüştür
  kalite yetersiz -> süreci iyileştir ve tekrarla
```

6.4 Standartlar

Hoca, daha önceki derslerde yazılım kalitesini sağlamak için yapılan aktivitelerden birinin standartların kullanımı olduğunu, hem kurum standartlarının hem proje standartlarının bulunması gerektiğini söylediğini hatırlattı.

- Ürün standartları: yazılım ürününün sağlaması gereken özellikler ve belge formatları.
- Süreç standartları: yazılım geliştirme sürecinin nasıl yürütüleceğini tanımlayan standartlar. Örnekler: gereksinim spesifikasyon standardı, tasarım standardı, konfigürasyon yönetim planı standardı, test planı standardı. Bunların hepsi sürecin nasıl gerçekleştirileceğini anlatır. Örneğin değişiklik kontrol sürecinin nasıl yürütüleceği bir süreç standardıdır, bir belgenin formatı ise ürün standardıdır.

Standartların bazı sorunları da vardır: Süreci daha bürokratik ve daha tanımsal hale getirirler.

ISO 9000 kalite yönetimi için bir çerçeve oluşturur. ISO 9001'de ise kurumlarda geliştirme ve bakım süreçlerine nasıl uygulanacağını anlatan bir kalite sistemi tanımı vardır. Bunların yanında IEEE standartları vardır. Bunlar arasında belgelemenin nasıl yapılacağını belirten belgeleme standartları bulunur.

6.5 Kalite Planı ve Test Planı

Kalite planı kurumun kalite özelliklerini, kalite değerlendirme sürecini ve kurumsal standartlarını belirtir. Ayrıca ürün planları, süreç tanımları, kalite hedefleri ve risk yönetiminin nasıl olacağı da kalite planında yer alır. Kalite planı test planından farklıdır: Kalite planı kurum bazında daha genel bir süreci anlatır. Test planı ise her proje ve projenin alt ürünleri için hangi özelliklerin sağlanacağını gösterir.

6.6 Kalite Özellikleri

Slaytta konuya ISO çerçevesinden bakıldığı için kalite özellikleri ISO 9126 ile ilişkilendirilebilir. Hoca McCall'ın da aynı özellikleri söylediğini, McCall'da bunların üç gruba ayrılarak tanım-

landığını, slaytta ise ayırmadan verildiğini belirtti. Slayttaki kalite özellikleri şunlardır: güvenlik, güvenilirlik (reliability), sağlamlık (robustness), anlaşılabilirlik, test edilebilirlik, uyarlanabilirlik, modülerlik, karmaşıklık, yeniden kullanılabilirlik, kullanışlılık, taşınabilirlik ve etkinlik (verimlilik).

7. Yazılım Ölçümü ve Metrikler

7.1 Neden Ölçüyoruz?

Kaliteyi kontrol edebilmek için ölçmek gerekir. Bunun bir yolu kalite gözden geçirmeleridir (quality review): programların, belgelerin ve ilerlemenin standartlara uygun olup olmadığının gözden geçirilmesi. Diğer yolu yazılım ölçümü ve metrikleridir. Ölçümler hem ürünle hem süreçle ilgili olabilir, farklı teknikler kullanılabilir ve ölçüm sistematik olarak yapılabilir.

Yazılım metriği, yazılımın herhangi bir özelliğinin ölçülebilmesi için kullanılan birimdir. Örneğin büyüklük için bir programdaki kod satırı sayısı (lines of code), çaba için kişi-gün sayısı (number of person-days), yani kaç kişinin kaç gün çalıştığı kullanılır.

Yazılım süreci içinde bir yazılım ürünü geliştirilir. Bu ürün üzerinde ölçümler yapılabilir. Süreç de süreç ölçümleriyle kontrol edilebilir. Hem ürünle hem süreçle ilgili ölçümlerin sonucunda yönetim bazı kararlar verir ve yazılım geliştirme sürecinde değişiklikler olur.

> **Hocanın vurgusu:** Ölçemediğinizi kontrol edemezsiniz.

7.2 İç ve Dış Özellikler

Metriklerde bazı kabuller vardır. Önce neyi bilmek istediğimizi, sonra neyi ölçeceğimizi belirleriz ve bu ikisi arasında bir ilişki kurarız. Yazılımın dışarıdan bakıldığında görülen özellikleri ile iç özellikleri arasındaki bu ilişki belirlenmelidir. Yani önce kalite özellikleri seçilir, sonra bunlar ölçülmeye çalışılır.

Örneğin bakım yapılabilirlik (maintainability) bizim kalite özelliklerimizden biridir, ama doğrudan ölçülemez. Neye bağlıdır? Prosedürlerdeki parametre sayısına, çevrimsel karmaşıklığa (cyclomatic complexity), yani programın ne kadar karmaşık olduğuna, programın ne kadar büyük olduğuna (kod satırı sayısı) ve kullanıcı el kitabının uzunluğuna. Bunların hepsi bakım yapılabilirlik hakkında bilgi verir.

Dış özellik (solda)	İlişkili iç özellikler (sağda)
Bakım yapılabilirlik	Prosedürlerdeki parametre sayısı, çevrimsel karmaşıklık, kod satırı sayısı, kullanıcı el kitabının uzunluğu
Güvenilirlik (ne kadar hatasız)	Çevrimsel karmaşıklık, program uzunluğu, programdaki hata mesajlarının sayısı
Taşınabilirlik (portability)	Parametre geçişleri (fazlaysa taşınabilirlik düşer), program uzunluğu
Kullanılabilirlik	El kitabının kapsamı, hata mesajlarının sayısı

Hoca, McCall kalite modelinde de görülen yazılım kalite özellikleri ile ölçülebilir değerlerin ilişkilendirilmesinin önemli olduğunu söyledi. Tablonun solundakilere dış, sağındakilere iç kalite özellikleri denir.

7.3 Ölçüm Süreci

Ölçüm süreci, bu ölçülere göre değerlendirme yapmaktır:

```
Yapılacak ölçümleri seç -> Değerlendirilecek bileşenleri seç
  -> Bileşen özelliklerini ölç -> Aykırı ölçümleri belirle
  -> Aykırı bileşenleri analiz et
```

Bir metrik hazırlamak için ürün ve süreç verisi toplamak gerekir ve bu verinin mümkünse otomatik olarak toplanması gerekir. Statik ürün analizi, dinamik ürün analizi ve süreç verisi

hakkında bilgi toplanır. Sonra bu verinin doğruluğu üzerinde çalışılır ve ürün metrikleri ortaya çıkarılır.

- Dinamik metrikler: programın çalıştırılması sonucunda elde edilir. Sistemin etkinliğini ve güvenilirliğini değerlendirir.
- Statik metrikler: sistemin gösterimlerinden (kod, tasarım) toplanan verilerle elde edilir. Karmaşıklık, anlaşılabilirlik ve bakım yapılabilirlik açısından değerlendirme yapar.

Hoca ekranı internet kopukluğu nedeniyle yeniden paylaştıktan sonra metrikler konusuna devam etti. Örneğin birimler arasındaki ilişki, yani bağlaşım (coupling), yapısal programlamada modülerlikle ilişkilidir. Nesneye yönelik yaklaşımda ise nesneler ve farklı sınıflar arasındaki ilişkiyle ilgilidir. Bu yüzden nesneye yönelik metrikler ile yapısal programlama metrikleri aynı isimli özelliklerle ve tanımlarla ifade edilebilir. Ölçüm analizinde, ölçümler sırasında beklenmedik sonuçlar (sürprizler) çıkabilir. Hoca bu konuların daha önce işlendiğini, burada farklı ifadelerle yeniden görüldüğünü söyledi.

8. Konfigürasyon Yönetimi

8.1 Hedef ve Kapsam

Konfigürasyon yönetimi, yazılım kalitesini sağlamak için yapılan aktivitelerden biridir ve genel kalite yönetimi sürecinin bir parçasıdır. Hoca daha önce hep temel bilgiler verdiğini, burada konuyu biraz daha ayrıntılı göreceklerini söyledi. Konfigürasyon yönetiminin konuları şunlardır: konfigürasyon yönetim planlaması, değişiklik yönetimi (change management), versiyon ve sürüm yönetimi (geçen hafta üzerinde durulmuştu) ve konfigürasyon yönetimini destekleyen CASE araçları. Yazılım sisteminin yeni versiyonları oluştukça bunların takip edilmesi, özel kullanıcı gereksinimlerinin takip edilmesi ve sistemde değişiklik olduğunda bu değişikliklerin kayda geçirilmesi gerekir.

8.2 Baseline

Baseline, sürecin belli bir aşamasında hangi temel ürünlerin bulunduğunu gösterir. Yani "süreç içinde hangi temel aşamada, ne durumdayız, hangi yazılım elemanlarımız var?" sorusunun cevabıdır.

Aşama sonu	Baseline'daki temel ürünler
Gereksinim analizi	SRS belgesi, proje planı
Tasarım	Sistemi anlatan diyagramlar, veri tabanı tasarım diyagramları (ER diyagramları)
Gerçekleştirme (kodlama)	Her modülün kodu, modüllerin birim testinde kullanılan test case'ler

Sistem üzerinde her değişiklik yapıldığında bu da ayrı bir baseline kabul edilir. Sistem ailelerinden de söz edilebilir: başlangıç sistemi ile bunun farklı bilgisayarlarda ve farklı işletim sistemlerinde çalışan versiyonları.

8.3 Konfigürasyon Yönetimi Standartları ve Planı

Konfigürasyon yönetimi standartları, bir kurumda her versiyonda takip edilecek standartlardır. Konfigürasyon yönetimi için bir IEEE standardından söz edilebilir. Standartlar yeni bir konfigürasyon elemanının nasıl tanımlanacağını da belirler. Hoca geliştirme ile testin birbiriyle ilişkili olduğunu, ikisinin eş zamanlı da yürüyebileceğini ve yeni bir versiyonun nasıl test edileceğinin de belirlenmesi gerektiğini söyledi.

Büyük bir yazılım sisteminde binlerce belge üretilir: planlar, spesifikasyonlar, tasarımlar, programlar, test verisi ve kullanıcı el kitapları. Konfigürasyon yönetim planı şunları belirtir:

- Belgelerin isimlendirme şeması (hiyerarşik isimlendirme).
- Konfigürasyon yönetim süreçlerinin sorumluluğunu kimin alacağı.
- Değişiklik kontrolünün ve versiyon yönetiminin kuralları.

- Bakımın nasıl yapılacağı.

Bir kod, bir test verisi, bir veri tabanı tablosu birer konfigürasyon maddesidir (configuration item). Büyük projelerde bunların hepsi tanımlanırken binlerce belge oluşur. Bunlar sonradan ayrı ayrı tanımlanmaz: Tanım yapılırken bir yandan da konfigürasyon yönetim yazılımına bu eleman hakkında bilgi girilir. Böylece bir veri tabanı oluşur. Konfigürasyon yönetim veri tabanında yazılım elemanlarının her tipi hakkında tutulacak bilgiler bulunur. Bu veri tabanı, elemanlar yaratılırken bir CASE aracıyla doldurulur. Hoca, büyük projelerde bilgiyi sonradan girmenin pek mümkün olmadığını söyledi.

8.4 Değişiklik Yönetimi

Konfigürasyon maddeleri üzerinde değişiklik gerekebilir. Değişiklik istekleri geliştiricilerden, kullanıcılardan ya da pazardaki zorlamalardan gelebilir. Değişiklik yapıldığında değişikliği kimin yaptığı ve değişikliğin ne olduğu konfigürasyon yönetim aracına aktarılmalıdır. Kitapta değişiklik yönetimi süreci bir algoritma biçiminde verilmiştir:

```
Değişiklik isteğini (formu) tamamla
  -> Değişiklik isteğini analiz et
  -> İstek geçerliyse: nasıl değiştirileceğini bul, değişiklik maliyetini hesapla
  -> Maliyete göre değişikliğin yapılıp yapılmayacağına karar ver
  -> Kabul edilirse: değişikliği yap, yeni bir sistem versiyonu yarat
```

Değişiklik istek formu, yazılım geliştirmeye başlandığı andan itibaren kullanılabilir. Geliştirme sürecinde de değişiklik istenebilir, ama sonradan istenen değişiklikler daha önemli ve tabii ki daha maliyetlidir. Değişikliğin ne zaman yapıldığı ve değişiklik isteğinin kapanıp kapanmadığı izlenir. Böylece yeni bir versiyona, yeni bir sürüme geçilir. Yeni sistem sürümünün nasıl dağıtılacağının yönetilmesi ve planlanması gerekir.

8.5 Versiyon ve Sürüm Yönetimi

Versiyon, işlevsel olarak diğerlerinden farklılık getiren bir sistem örneğidir. Sürüm (release) ise geliştirme takımının kullanıcılara dağıttığı versiyondur. Versiyonun nasıl tanımlanacağı, versiyon numaralandırmasının ve değişiklik tanımının nasıl yapılacağı belirtilmelidir.

Numaralandırmada küçük değişikliklerde 1.1, 1.2 gibi, daha önemli, gerçek bir değişiklikte 2.0, 2.1 gibi numaralar verilebilir. Slaytta versiyon numaralarının oluşturduğu bir türetme ağacı gösterildi. Numara yerine özellik tabanlı (attribute-based) ya da değişiklik tabanlı (change-oriented) tanımlama da kullanılabilir.

Sürüm yönetimi geçen hafta işlenmişti. Yeni ya da yenilenmiş ürünün kullanıcılara nasıl aktarılacağı, kullanıma alma yolları ve geçiş planlaması bu konunun içindedir. Sürümlerdeki problemler şunlardır:

- Eski sürümden yeni sürüme geçişte aksaklık olmaması gerekir.
- Önceki sürümlerde kabul edilmiş kullanım süreci çok değişiyorsa bir kabul ve alışma süreci geçer.
- Aktarmanın veri kaybı olmadan yapılması gerekir.

Hoca sürümde hem teknik kalitenin hem pazarlama tarafının önemli olduğunu söyledi. Sürüm stratejisini belirleyen etkenler şunlardır: sistemin teknik kalitesi, platform değişiklikleri ve kapsamı, pazarlama gereksinimleri, müşterinin değişiklik önerilerinin olup olmadığı.

Sistem oluşturma (system building), bileşenlerin paket haline getirilmesi ve bütünleştirilmesinin nasıl yapılacağına karar verme aşamasıdır. Konfigürasyon yönetimi için CASE aracı kullanılmalıdır. Bu araçlar genellikle proje yönetim araçlarının içindedir. Yalnız değişiklik yönetimi için kullanılan versiyon yönetim sistemleri de vardır. Hoca bunların üzerinde durmadı.

> **Ek bilgi (derste anlatılmadı):** Günümüzde yalnız versiyon yönetimi için kullanılan sistemlere büyük olasılıkla en bilinen örnek Git'tir; daha eski örnekler arasında CVS ve Subversion sayılır. Bu araçlar her değişikliği kimin, ne zaman ve neden yaptığını kaydederek konfigürasyon yönetiminin kayıt tutma işini otomatikleştirir.

9. Süreç İyileştirme ve CMMI

9.1 Süreç İyileştirme ve Süreç Özellikleri

Süreç iyileştirme (process improvement), yazılım geliştiren kurumların daha iyi hizmet vermesi için yapılan iyileştirmedir. Hoca daha önce örnek olarak CMMI'ı gördüklerini, burada da örnek olarak onun verildiğini hatırlattı. Süreç özellikleri şunlardır:

- Anlaşılabilirlik: her süreç tanımı belgeleriyle birlikte anlaşılıyor mu?
- Görünürlük: süreç işi yapanlarca izlenebilir, görünür mü?
- Desteklenebilirlik: CASE araçları kullanılmış mı?
- Kabul edilebilirlik: tanımlanan süreçler kabul edilebilir süreçler mi?
- Güvenilirlik (reliability): hata oranı ne kadar?
- Sağlamlık (dayanıklılık): beklenmeyen problemlere rağmen süreç sürdürülebiliyor mu? Yazılım geliştirmede çıkan problemlere karşı risk analizi yapılmış mı?
- Bakım yapılabilirlik ve takip edilebilirlik.
- Hız: yazılım ne kadar sürede geliştiriliyor?

Süreç iyileştirmenin aşamaları, derste daha önce benzeri çizilen şu döngüdür:

```
Süreci ölç -> Süreci analiz et -> Gerekirse süreçte değişiklik yap -> (tekrar ölç)
```

9.2 Süreç Kalitesi ve Ürün Kalitesi İlişkisi

Süreç kalitesi ile ürün kalitesi birbiriyle ilişkilidir. Ürün kalitesine bakıldığında şunlar etkilidir: ürün hangi teknolojiyle geliştirilmiş, geliştiren insanların kalitesi nedir, zaman, maliyet ve iş takibi açısından ürün nasıl geliştirilmiş ve süreç kalitesi. Süreç kalitesi ise bütün yapılması gereken işlemlerin yapılıp yapılmadığıdır. Hoca bunu "tasarımı atlayıp doğrudan kodu üretmeye mi başlamışlar?" sorusuyla örnekledi.

Kalite faktörleri dendiğinde sürecin yetkinliğinden, yani yönetilebilir ve desteklenebilir olmasından söz edilir. Küçük sistemlerde ve prototiplerde informal süreçler yeterli olabilir. Büyük sistemlerde ise yönetilebilen ve daha iyi anlaşılabilen süreçler gerekir.

Süreç araç desteği de incelenir: Yazılım geliştirme süreci boyunca temel aktivitelerde araç kullanılmış mı? Konfigürasyon yönetim aracı, proje yönetim aracı, analiz ve tasarım platformları kullanılmış mı?

9.3 Süreç Ölçümü ve Süreç Modelinin İncelenmesi

Süreç ölçümünde önce hedefler belirlenir. Sonra bu hedefleri değerlendirecek sorular ve ölçüm birimleri seçilir, süreç analiz edilir ve buna göre modellenir.

Süreç modellerinde standartlar, sorgular ve gözden geçirmeler yapılabilir. Süreç modeli incelenirken bakılan elemanlar şunlardır: aktiviteler, süreçler, dağıtılan ara ürünler ve son ürünler (deliverables), süreçteki koşullar, kişilerin rolleri, sorumlulukları ve iletişimleri. Bunların hepsi yazılım geliştirme sürecini gözden geçirmeyi sağlar. Örneğin bir modül testi aktivitesinde ön koşullar sağlandı mı, test mühendisi rolündekiler ne yaptı, test modüllerini oluşturdular mı gibi sorular sorulabilir.

9.4 CMMI

Süreci iyileştirmek için kullanılan çerçevelerden biri CMMI'dır (Capability Maturity Model Integration). Software Engineering Institute (SEI) tarafından 1990'larda CMM, 2000'lerde CMMI kullanılmaya başlandı. İlk CMM modelinin düzeyleri başlangıç (initial), tekrarlanabilir (repeatable), tanımlı (defined), yönetilen (managed) ve iyileştiren (optimizing) düzeylerdir. Bu modelle problemler yaşandı: Farklı modellerin her biri ayrı ayrı yürütülüyordu. CMMI'da bunlar bütünleştirildi ve süreç alanları, hedefler ve pratikler oluşturuldu. Süreç yönetimi, proje yönetimi, mühendislik ve destek alanları CMMI'da birleştirildi.

CMMI'ın hedefleri arasında projeyi doğru izlemek ve kontrol etmek, tüm gereksinimleri analiz etmek, gereken işlevlerin geliştirilmesini sağlamak ve genel hedeflere uygun bir çalışma durumu göstermek vardır. Hocanın sıraladığı CMMI düzeyleri şöyledir:

Düzey	Ad
0	Eksik (incomplete)
1	Gerçekleştirilmiş (performed)
2	Yönetilen (managed)
3	Tanımlı (defined)
4	Nicel yönetilen (quantitatively managed)
5	İyileştiren (optimizing)

Hoca ayrıca gereksinim yönetimi, proje planlama, izleme, ölçüm ve analiz süreç alanlarının "birinci aşamada" bulunduğunu söyledi ve düzeylerin hangi süreç alanlarını içerdiğinin modelde belirtildiğini ekledi.

10. Ders Sonrası Notlar

Ders sonrası not: Hoca "ISO 9000 alt çerçevesinde 9100-9126 gibi kalite tanımları" dedi. ISO/IEC 9126 büyük olasılıkla ISO 9000 ailesine ait değildir; ayrı bir yazılım ürün kalitesi standardıdır ve yerini büyük olasılıkla 2011'de ISO/IEC 25010 almıştır.

Ders sonrası not: Hoca CMMI düzeylerini 0'dan 5'e sıraladı. Bu sıralama büyük olasılıkla CMMI'ın sürekli (continuous) gösterimindeki yetenek düzeylerine aittir. Aşamalı (staged) gösterimde olgunluk düzeyleri büyük olasılıkla 1-5'tir: initial, managed, defined, quantitatively managed, optimizing.

Ders sonrası not: Hoca gereksinim yönetimi, proje planlama, izleme, ölçüm ve analizin "birinci aşamada" olduğunu söyledi. Aşamalı CMMI'da bu süreç alanları büyük olasılıkla 2. düzeye (managed) aittir; 1. düzeyde tanımlı süreç alanı yoktur.

Ders sonrası not: Hoca denklik bölümlemesinde girdilerin "eşit aralıklara" bölündüğünü söyledi. Denklik sınıflarının eşit genişlikte olması gerekmez; sınıflar, sistemin aynı biçimde davranması beklenen değer kümelerine göre belirlenir. Slayttaki 10000-99999 aralığı ile 10000'den küçük değerler de eşit genişlikte değildir.

Ders sonrası not: Altyazıda "Summerville" diye geçen Sommerville, "Clearroom" diye geçen cleanroom, "MacColay", "MacCol" ve "Makkol" diye geçen McCall, "Ecolons Partitioning" diye geçen equivalence partitioning, "Yüz kez/Yuz kez" diye geçen use case, "data seconds diagramı" diye geçen sequence diyagramı, "SMM/SMMI" diye geçen CMM/CMMI, "capability majority model" diye geçen capability maturity model'dır.

11. Örnek Sorular

Bu derste ekran görüntüsü kullanılmadı. Aşağıdaki sorular hocanın tekrar sırasında öğrencilere sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:36:11): İnceleme (inspection) ne tür bir işlemdir? Validation mıdır, verification mıdır?

Çözüm (derste verildi): İnceleme bir doğrulamadır (verification), daha açık söylemek gerekirse statik doğrulamadır (static verification). Program çalıştırılmadan, gerekirse araç desteğiyle, ürünün spesifikasyona ve standartlara uygunluğuna bakılır.

Soru 2 (derste sözlü, 00:36:28): Dinamik doğrulama (dynamic verification) nedir?

Çözüm (derste verildi): Dinamik doğrulama testtir. Ürün çalıştırılarak davranışı gözlenir.

Dersin Toparlaması

Bu derste dönemin konuları Sommerville'in kitabı üzerinden bir bütün olarak yeniden gözden geçirildi. Doğrulama "ürünü doğru yolla mı yapıyoruz", geçerleme "doğru ürünü mü yapıyo-

ruz” sorusuna cevap verir. İnceleme statik, test dinamik doğrulamadır ve ikisi birbirinin alternatifi değil, tamamlayıcısıdır. Testte kusur testi hata bulmayı, geçerleme testi gereksinimlerin karşılandığını göstermeyi hedefler. Testler birimden kabul testine doğru ilerler ve V modeliyle geliştirme aşamalarına bağlanır. Test case’ler gereksinim tabanlı, bölümleme ve yapısal yaklaşımlarla tasarlanır. İkili arama örneğinde temel yol sayısı kapalı bölge sayısına bir eklenerek bulunur.

Dersin ikinci yarısında kalite yönetimi (kalite güvencesi, planlama, kontrol, süreç ve ürün kalitesi, standartlar), ölçüm (“ölçemediğinizi kontrol edemezsiniz”, iç ve dış özellikler), konfigürasyon yönetimi (baseline, plan, değişiklik, versiyon ve sürüm yönetimi) ve süreç iyileştirme (ölç, analiz et, değiştir döngüsü ve CMMI) tekrar edildi. Bu ders, sonraki haftalarda grupların sunacağı test raporlarında kullanılacak kavramların toplu bir özetidir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.